HELIOS

Sistema unificado de citas medicas

Sprint 1 - Daily Scrum

Confirmacion de cita medica end-to-end

Grupo HELIOS

Jose Brown (PO) - Ethan Gonzalez (Frontend) - Alejandro Saenz (Tech Lead)

Ashley Martinez (QA) - Marian Barba (Scrum Master / DevOps)

UTP - Calidad de Software - 2026 - 3 minutos

DAILY SCRUM

Respuesta a las 3 preguntas obligatorias

Que hicimos esta sprint?

- 5 historias de usuario (US-01 a US-05) al 100%
- Dashboard medico desplegado en produccion
- Seguridad multicapa con RLS por rol
- 65 tests automatizados PASS
- 20 bugs detectados y resueltos

Que haremos en Sprint 2?

- Cancelar cita (paciente autoservicio)
- Reprogramar cita a otro horario
- Tabla citas_historial para auditoria
- Tests E2E con Playwright
- Locust a 300 usuarios

Que bloqueos tuvimos?

- Bug B-20 race React (4 iteraciones)
- Detectado en uso real por PO
- Spinner >10s tras F5 (B-15+B-17)
- Cero pendientes al cierre del sprint
- Resueltos con Plan Mode + tests

DEMO EN VIVO

Escanea el QR y pruebalo en tu celular



helios-web-production.up.railway.app

1. Registrate con un correo nuevo (ej: prueba@gmail.com)
2. Elige especialidad y sede
3. Selecciona medico de la lista
4. Elige horario en el calendario
5. Confirma con tu cedula panamena
6. Recibe tu codigo de cita unico

QUE HIZO CADA UNO

Aporte individual de cada integrante

Jose Brown

Product Owner

- Validacion de cada deploy real
- Deteccion 8 bugs criticos en uso
- Lidero las 4 iteraciones del B-20
- Priorizo backlog del sprint

Marian Barba

Scrum Master / DevOps

- Setup inicial del proyecto
- Deploys CI/CD a Railway (26 commits)
- Configuracion 5 agentes IA
- Coordinacion ceremonias Scrum

Ethan Gonzalez

Dev Frontend

- Componentes React + TypeScript
- Fix B-05 (toolbar) y B-08 (hooks)
- Vista mobile del calendario
- Integracion con Supabase Auth

Alejandro Saenz

Tech Lead

- Arquitectura Supabase + RLS
- Funcion SQL crear_cita_atomica
- Fix B-09 (seguridad sessionStorage)
- Decisiones tecnicas del sprint

Ashley Martinez

QA Engineer

- Pruebas funcionales del wizard
- Documentacion de metricas (6 commits)
- Casos de prueba para el demo
- Calculo tasa defectos 8.70 bugs/KLOC

FLUJO DEL USUARIO

Como navega un paciente / medico por el sistema

Rutas principales:

Paciente nuevo:

Registro -> Login -> Agendar

Paciente existe:

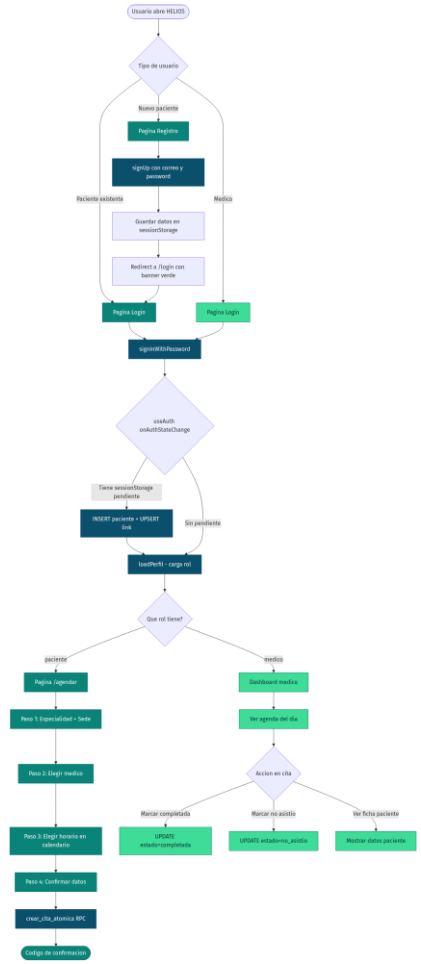
Login -> Agendar / Mis Citas

Medico:

Login -> Dashboard agenda hoy

Race React fix:

INSERT en useAuth listener



METRICAS ISTQB

Lo que medimos segun el estandar internacional

65

Tests automatizados

Vitest + RTL. Todos pasan.
Threshold 60% activo en CI.

61.4%

Cobertura de codigo

Statements 61.4% - Functions 90.9%.
Build falla si baja de 60%.

300 ms

Latencia P95

Locust con 200 usuarios concurrentes.
Meta era 2s. Lo superamos 6.6x.

20

Defectos resueltos

Tasa 8.70 bugs/KLOC (rango profesional).
Cero pendientes al cierre.

SONARCLOUD LIVE

Analisis de calidad y seguridad en vivo

Vamos a abrir SonarCloud y mostrar:

- **Bugs detectados** — Issues que SonarCloud encontro automaticamente
- **Code smells** — Patrones que podrian causar problemas
- **Vulnerabilidades** — Problemas de seguridad detectados
- **Coverage** — % de codigo cubierto por nuestros tests
- **Duplication** — Codigo duplicado que se debe refactorizar
- **Quality Gate** — Pasa o falla el threshold de calidad



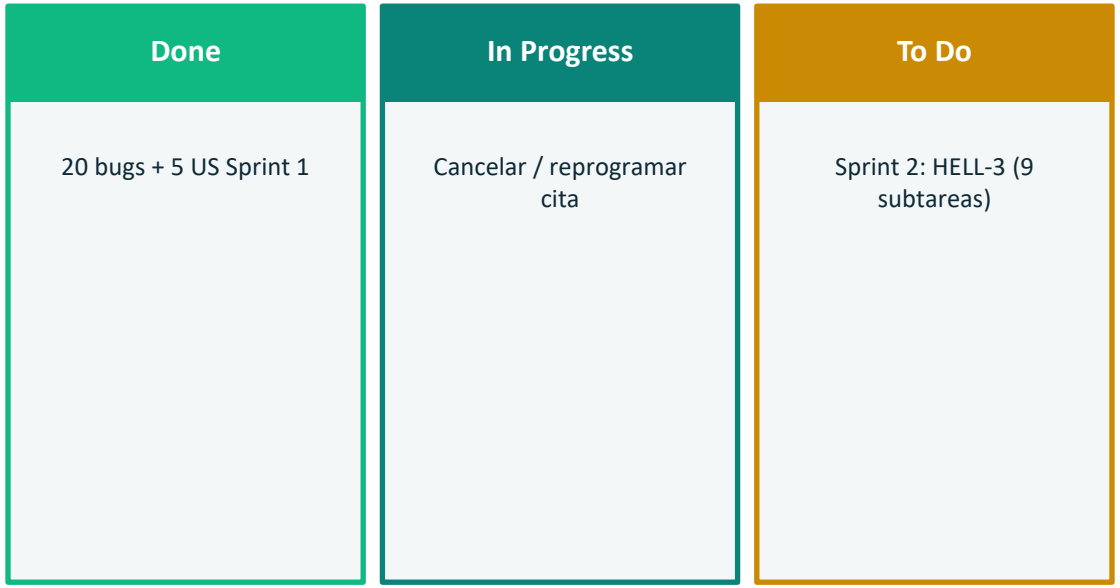
CIERRE

Herramientas + Kanban + Preguntas

7 herramientas usadas

- **GitHub** Repositorio + 80+ commits
- **Jira** Tablero Kanban 21 issues
- **Vitest** 65 tests automatizados
- **Locust** P95 300ms con 200 users
- **Supabase** BD + Auth + API
- **Railway** Deploy CI automatico
- **SonarCloud** Calidad + seguridad (NUEVO)

Tablero Kanban Jira



MVP funcional - Metricas medidas - Proceso documentado

Gracias. Preguntas?